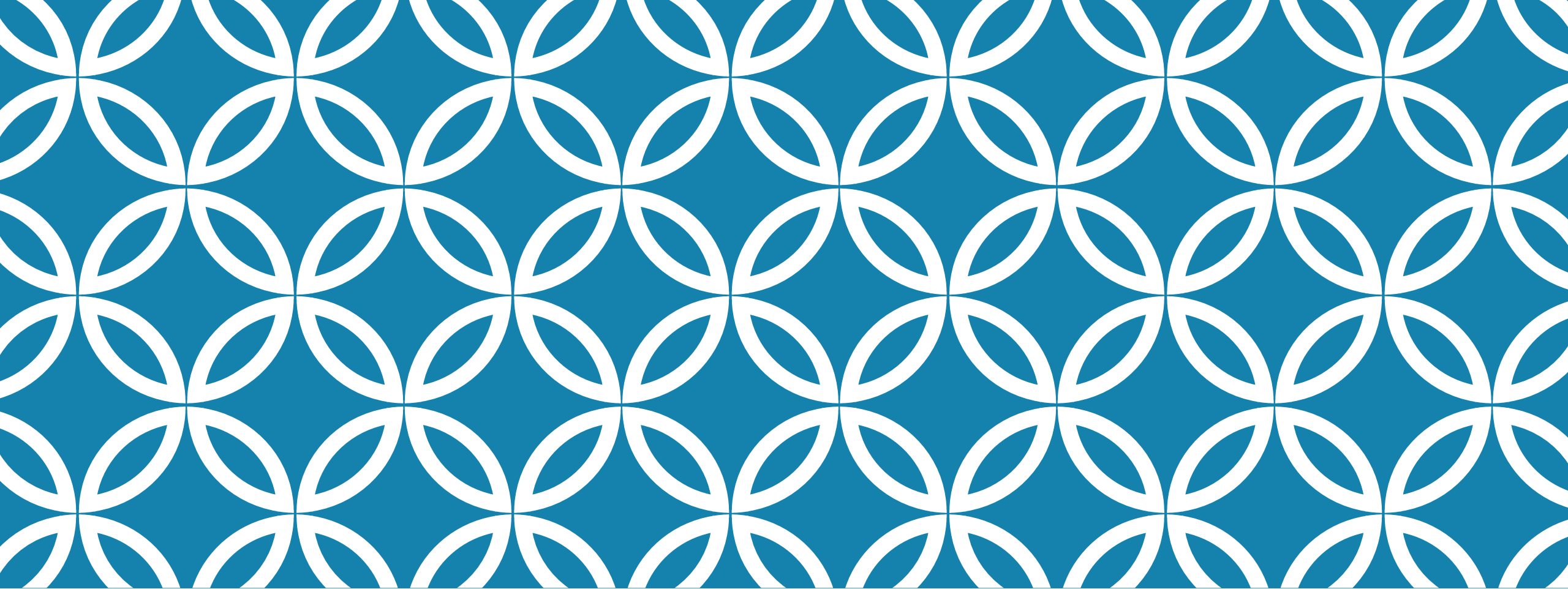
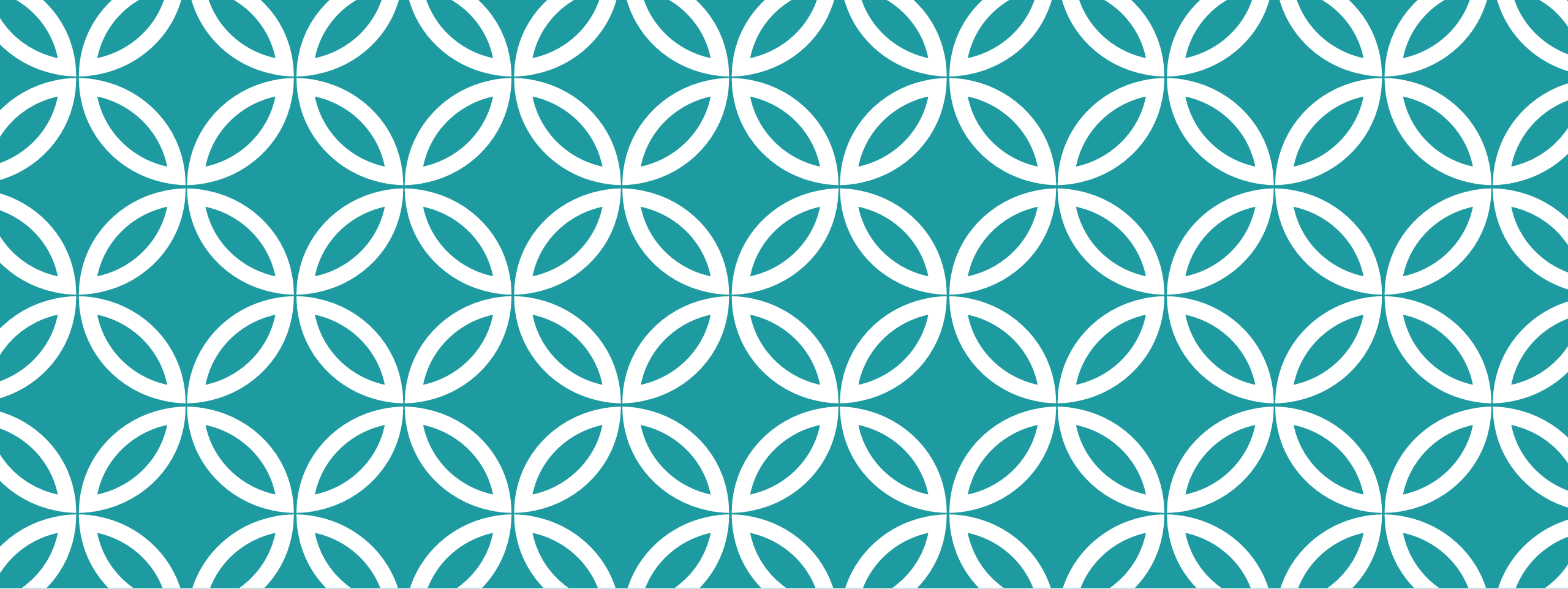




RCPP 01

dr Maria Kubara, WNE UW



CRASH COURSE IN (R)C++

Minimalistic OOP

WHY?

C++ is faster than R

Helps to solve bottlenecks like:

- Loops (that cannot be vectorized)
- Recursive functions

FUNCTION IN C++

- ❖ Give set of inputs (function arguments)
- ❖ Run some code on them
- ❖ Return a single object

DIFFERENCES

Every command must be terminated with a **semicolon ;**

We must **declare types** of objects once we start working with them (initializing a variable requires declaring its type). In particular:

- Types of function arguments
- Type of returned value
- Type of any intermediate objects created inside of the function

C++ functions must have a clear return statement -> once function encounters first return statement – it will exit

Assignment operator in C++ is = (not an arrow)

SIMILARITIES

Syntax of `if()` statement is the same in both languages

While loop has the same syntax as well

Like in R, in C++ we can use a command to skip given loop repetition (in R this is called *next*, in C++ this is called *continue*)

Comparison operator `==` is the same in both languages

The same for operator `!=`, `>`, `<`, etc.

DIFFERENCES

Comments in C++ will be written as follows:

- one-line comment can be created using `// ...`
- multiline comments are created using `/*...*/`

Assignment operator in C++ is `=` (not an arrow)

C++ provides operators modifying a value in place:

- eg `i++` increases the value of `i` by 1.
- Similar modifying operators in place are `-=`, `*=`, `/=`.

To find the length of a vector in C++, we use the `.size()` method (returns an integer)

DIFFERENCES

The `for()` statement has different syntax in C++:

- `for (init; check; increment)`
- After each iteration we have to increase the value of `init`
- Loop is executed as long as the “check” argument is met

- Example: **`for(int i=0; i<n; i++)`**
- start with `i = 0`,
- execute until `i` is smaller than `n`,
- after each execution increase `i` by one (inplace increment)

in C++ indexing vectors starts with 0 (similarly to python)

CLASSES IN C++ (SINGLE VALUES — SCALARS)

C++ equivalents of individual values (scalar):

- R numeric -> double
- R integer -> int
- R character -> String
- R logical -> bool

CLASSES IN C++ (VECTORS)

C++ classes for the most popular types of R vectors are:

- `NumericVector` - vector of numerical values (floating point)
- `IntegerVector` - vector of integer values
- `CharacterVector` - vector of character (text) values
- `LogicalVector` - vector of logical value

Indexing vector[i]

INDEXING STARTS WITH 0

CLASSES IN C++ (MATRIX)

- NumericMatrix
- IntegerMatrix
- CharacterMatrix
- LogicalMatrix

Indexing: `x(i, j)`

`myMatrix.nrow()` and `myMatrix.ncol()` give you dimensions of matrix

EXAMPLE

```
bool myFunction(double x) {  
    bool checker = true;  
    if (x > 0) {  
        return checker;  
    } else {  
        checker = false;  
        return checker;  
    }  
}
```

EXAMPLE

We specify name of our function



```
bool myFunction(double x) {  
    bool checker = true;  
    if (x > 0) {  
        return checker;  
    } else {  
        checker = false;  
        return checker;  
    }  
}
```

EXAMPLE

We specify what class of an object our function will return
Here: bool -> logical type



```
bool myFunction(double x) {  
    bool checker = true;  
    if (x > 0) {  
        return checker;  
    } else {  
        checker = false;  
        return checker;  
    }  
}
```

EXAMPLE

We specify arguments of our function (if more than one, use a comma)

We need to specify their type as well (here double -> numeric)



```
bool myFunction(double x) {  
    bool checker = true;  
    if (x > 0) {  
        return checker;  
    } else {  
        checker = false;  
        return checker;  
    }  
}
```

EXAMPLE

```
bool myFunction(double x) {  
    bool checker = true;  
    if (x > 0) {  
        return checker;  
    } else {  
        checker = false;  
        return checker;  
    }  
}
```

We specify a new intermediate object
-> we need to specify its type as well



EXAMPLE

```
bool myFunction(double x) {
```

```
    bool checker = true;
```

```
    if (x > 0) {
```

```
        return checker;
```

```
    } else {
```

```
        checker = false;
```

```
        return checker;
```

```
    }
```

```
}
```

If else statement -> each step
is within a curly bracket { }



EXAMPLE

```
bool myFunction(double x) {  
    bool checker = true;  
    if (x > 0) {  
        return checker;  
    } else {  
        checker = false;  
        return checker;  
    }  
}
```

Every instruction is ended
with a semicolon



EXAMPLE

```
bool myFunction(double x) {  
    bool checker = true;  
    if (x > 0) {  
        return checker;  
    } else {  
        checker = false;  
        return checker;  
    }  
}
```

We have previously defined what type the “checker” will be. Now we can just modify its value, assigning a new one with = sign



EXAMPLE

```
bool myFunction(double x) {
```

```
    bool checker = true;
```

```
    if (x > 0) {
```

```
        return checker;
```

```
    } else {
```

```
        checker = false;
```

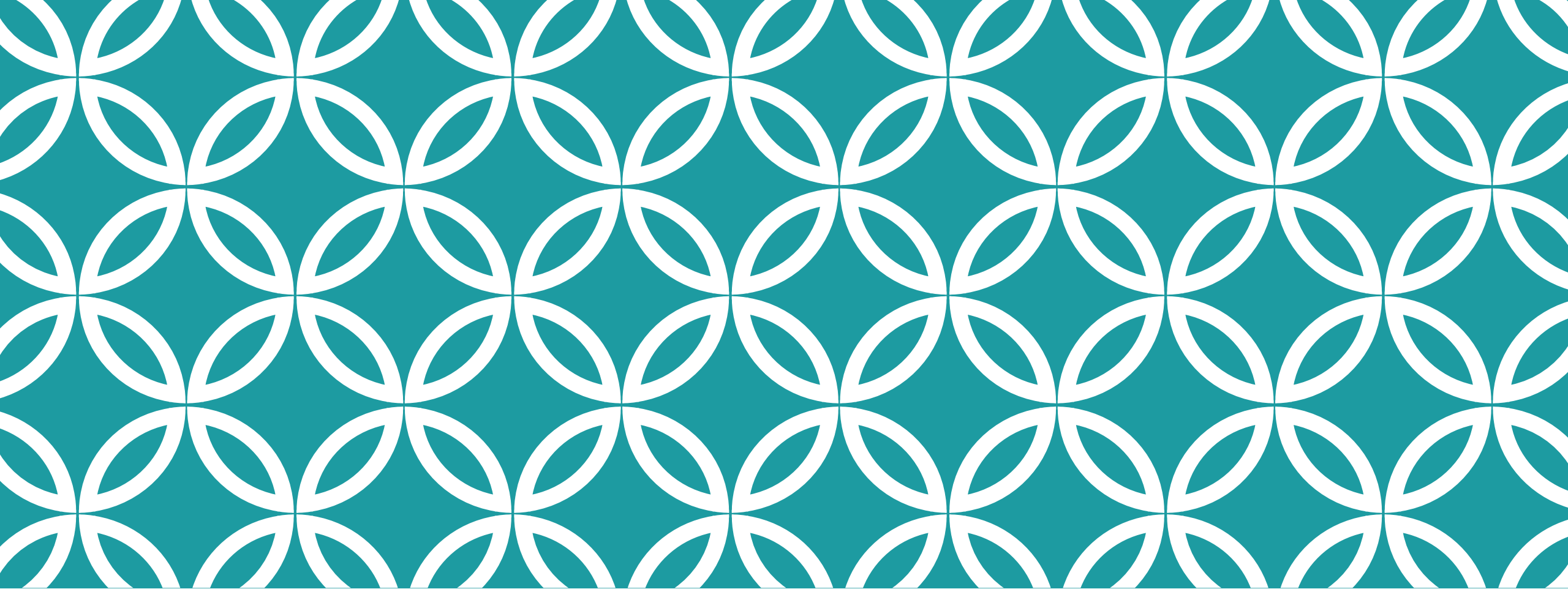
```
        return checker;
```

```
    }
```

```
}
```

Function must end with a return statement. It will return true if a given argument was positive and false in an opposite case.





WRITE C++ CODE IN R |

EMBEDDING C++ CODE IN R

cppFunction includes Rcpp namespace

```
cppFunction(  
  "int multiplyBy2(int x) {  
    Code int result = x * 2;  
    return result;  
  }"  
)
```

We can use C++ in R via cppFunction()

We will define C++ code as a character input to the cppFunction()

cppFunction() passes the code to the compiler and makes the function available in R environment

Side note: by default cppFunction() uses C++ namespace Rcpp

EMBEDDING C++ CODE IN R

```
cppFunction(  
  "int multiplyBy2(int x) {  
    int result = x * 2;  
    return result;  
  }"  
)  
  
# After compilation  
# (running previous chunk)  
#you can use it in R 😊  
multiplyBy2(5)
```

This function is named multiplyBy2.

It returns integer

It takes (single) integer input (numeric value will be coerces to an integer by dropping decimal part)

FUNCTION WITHOUT INPUT — RETURNING A SCALAR

```
tenR <- function() {  
  10L  
}
```

```
int tenC() {  
  return 10;  
}
```

To compile and use it →

put is an input to cppFunction()

Both of these functions return (integer) 10 after calling them in the console.

FUNCTION WITHOUT INPUT — RETURNING A SCALAR

```
tenR <- function() {  
  10L  
}
```

```
int tenC() {  
  return 10;  
}
```

To compile and use it →

put is an input to cppFunction()

Both of these functions return (integer) 10 after calling them in the console.

FUNCTION WITHOUT INPUT — RETURNING A SCALAR

```
tenR <- function() {  
  10L  
}
```

Notice the differences

```
int tenC() {  
  return 10;  
}
```

To compile and use it →

put is an input to cppFunction()

Both of these functions return (integer) 10 after calling them in the console.

INPUT AND OUTPUT IS A SCALAR

```
signR <- function(x) {  
  if (x > 0) 1 else  
    if (x == 0) 0 else -1  
}
```

Return a sign of the input:

1 for positive, 0 for zero and -1 for negative

```
int signC(double x) {  
  if (x > 0) {  
    return 1;  
  } else if (x == 0) {  
    return 0;  
  } else {  
    return -1;  
  }  
}
```

INPUT AND OUTPUT IS A SCALAR

```
signR <- function(x) {  
  if (x > 0) {  
    1  
  } else if (x == 0) {  
    0  
  } else {  
    -1  
  }  
}
```

Rewritten in lines to
match C++ more
directly

```
int signC(double x) {  
  if (x > 0) {  
    return 1;  
  } else if (x == 0) {  
    return 0;  
  } else {  
    return -1;  
  }  
}
```

INPUT AND OUTPUT IS A SCALAR

```
signR <- function(x) {  
  if (x > 0) {  
    return(1)  
  } else if (x == 0) {  
    return(0)  
  } else {  
    return(-1)  
  }  
}
```

```
int signC(double x) {  
  if (x > 0) {  
    return 1;  
  } else if (x == 0) {  
    return 0;  
  } else {  
    return -1;  
  }  
}
```

Or even better...

INPUT AND OUTPUT IS A SCALAR

```
signR <- function(x) {  
  if (x > 0) {  
    return(1)  
  } else if (x == 0) {  
    return(0)  
  } else {  
    return(-1)  
  }  
}
```

```
int signC(double x) {  
  if (x > 0) {  
    return 1;  
  } else if (x == 0) {  
    return 0;  
  } else {  
    return -1;  
  }  
}
```

Again – the differences

INPUT AND OUTPUT IS A SCALAR

```
signR <- function(x) {  
  if (x > 0) {  
    return(1)  
  } else if (x == 0) {  
    return(0)  
  } else {  
    return(-1)  
  }  
}
```

We know that the R function can only work on scalars because of the if() usage with == operator. It works for comparing single values only – in newer R version (like 4.2.2) it will throw an error when used on longer vectors

```
int signC(double x) {  
  if (x > 0) {  
    return 1;  
  } else if (x == 0) {  
    return 0;  
  } else {  
    return -1;  
  }  
}
```

See how type definition helps with understanding what the function does – it takes a single number – double x

In R it is not obvious whether x can be a longer vector or just a scalar (we need to check it in the documentation)

Both functions can work with longer vectors only when called within lapply() or purr::map()

VECTOR AS ARGUMENT, AND SCALAR AS RESULT

```
myMeanR = function(x) {  
  sum = 0  
  n = length(x)  
  for(i in 1:n)  
    sum = sum + x[i]  
  sum/n  
}
```

Calculate mean – with loop

```
double myMeanC(NumericVector x) {  
  double sum = 0;  
  int n = x.size();  
  for(int i=0; i<n; i++) {  
    sum = sum + x[i];  
  }  
  return sum/n;  
}
```


VECTOR AS ARGUMENT, AND SCALAR AS RESULT

```
myMeanR = function(x) {
```

```
  sum = 0
```

```
  n = length(x)
```

```
  for(i in 1:n)
```

```
    sum = sum + x[i]
```

```
  return(sum/n)
```

```
}
```

Again – the differences

```
double myMeanC(NumericVector x) {
```

```
  double sum = 0;
```

```
  int n = x.size();
```

```
  for(int i=0; i<n; i++) {
```

INDEXING FROM 0

```
    sum = sum + x[i];
```

```
}
```

```
  return sum/n;
```

```
}
```

Specifying end of loop
and increase of iterator

Loops are one of R bottlenecks – using them can lead to slow, unoptimal code.

Try to use vectorised operations/functions instead of loops -> or simply rewrite them to C++

RECURSIVE FUNCTION

```
fibor <- function(n) {  
  if (n <= 1) n else  
    fibor(n-1) + fibor(n-2)  
}
```

```
int fiboC(int n) {  
  if (n <= 1) return(n); else  
    return(fiboC(n-1) + fiboC(n-2));  
}
```

Recursive functions are calling themselves many times to get to the final results. They are one of the most problematic things in code optimisation across all languages. And yet, some languages are faster than others in terms of function call speed. Using recursive functions in C++ is much faster than in R (less overhead in calling the function and slower increase in time complexity).

RECURSIVE FUNCTION

```
fibor <- function(n) {  
  if (n <= 1){  
    return(n)  
  } else {  
    result = fibor(n-1) + fibor(n-2)  
    return(result)  
  }  
}
```

Again – the differences

```
int fiboC(int n) {  
  if (n <= 1) {  
    return n;  
  } else {  
    int result=fiboC(n-1)+fiboC(n-2);  
    return result;  
  }  
}
```

Compare vector of forecasts to a single real value – calculate how each model was wrong
– vector of error scores

VECTOR AS INPUT AND RESULT

function in R

```
releerrorR <- function(real,  
                        forecasts) {  
  abs(real - forecasts)/real  
}
```

It is not clear that this function takes single real value and works with a vector of forecasts

Function in R uses vectorisation instead of a loop – it is there
R truly shines 😊

```
NumericVector releerrorC(double real,  
                          NumericVector forecasts) {  
  int n = forecasts.size();  
  NumericVector releerror(n);  
  
  This implementation uses fast C++ loops  
  for(int i = 0; i < n; i++) {  
    releerror[i] = fabs(real - forecasts[i])/real;  
  }  
  return releerror;  
}
```

Compare vector of forecasts to a single real value – calculate how each model was wrong
– vector of error scores

VECTOR AS INPUT AND RESULT

function in R

```
relerorR <- function(real,  
                      forecasts) {  
  abs(real - forecasts)/real  
}
```

Important: these functions have similar efficiency
Vectorised code in R is as efficient as C++ loops

```
NumericVector relerrorC(double real,  
                        NumericVector forecasts) {  
  int n = forecasts.size();  
  NumericVector relerror(n);  
  Creating vector relerror of length n  
  
  for(int i = 0; i < n; i++) {  
    relerror[i] = fabs(real - forecasts[i])/real;  
  } Get absolute value of type double with fabs()  
  
  return relerror;  
}
```

MATRIX AS INPUT

Matrix in Rcpp is implemented by classes

NumericMatrix, CharacterMatrix,
LogicalMatrix, IntegerMatrix

We refer to rows and columns of matrix in R via **function** with (i,j) – bracket reference

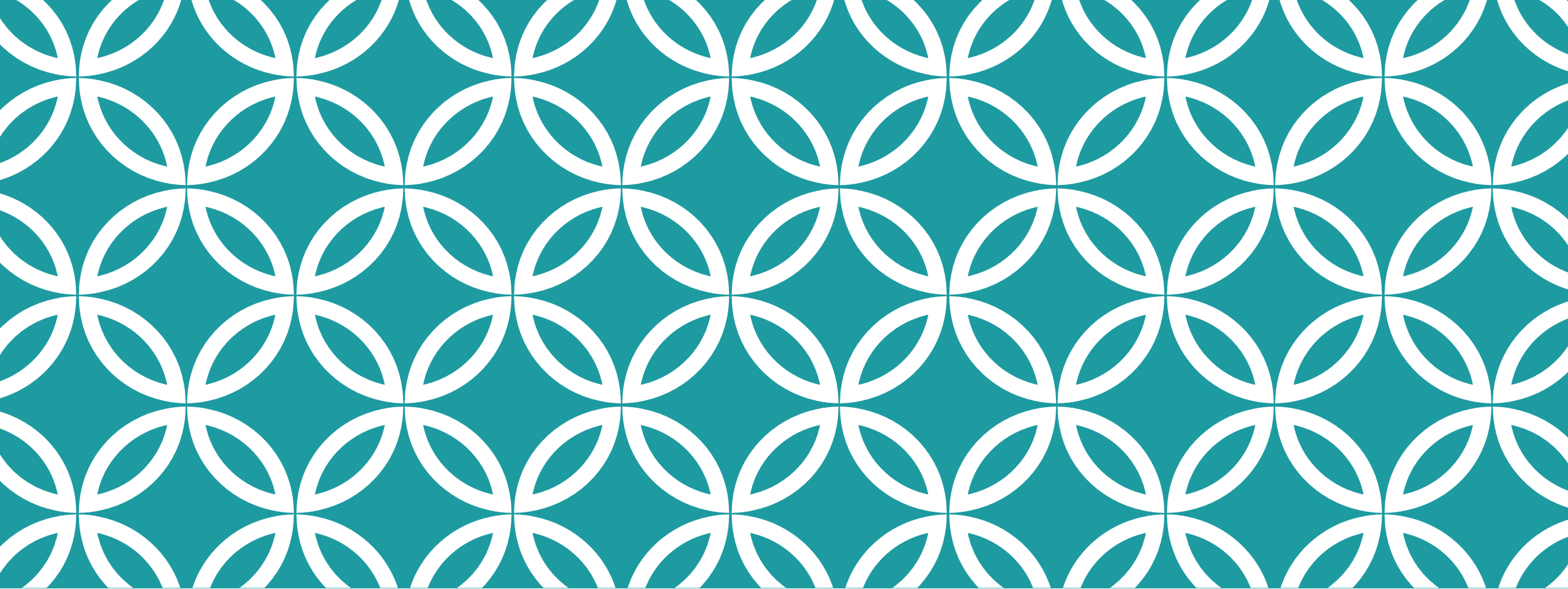
Remember to count from 0 in C++!

You can get dimensions of the matrix by using .nrow() and .ncol()

This function works as fast as built-in colmeans() function in R. Beware of the precision difference – double has different precision than numeric

```
cppFunction('NumericVector colMeansC(NumericMatrix x) {
    int nrow = x.nrow(), ncol = x.ncol();
    NumericVector colMeans(ncol);
    Create a vector colMeans of length ncol

    for (int j = 0; j < ncol; j++) {
        double total = 0;
        for (int i = 0; i < nrow; i++) {
            total += x(i, j);
            Modify position on the ith row and jth column
        }
        colMeans[j] = total/nrow;
    }
    return colMeans;
}')
```



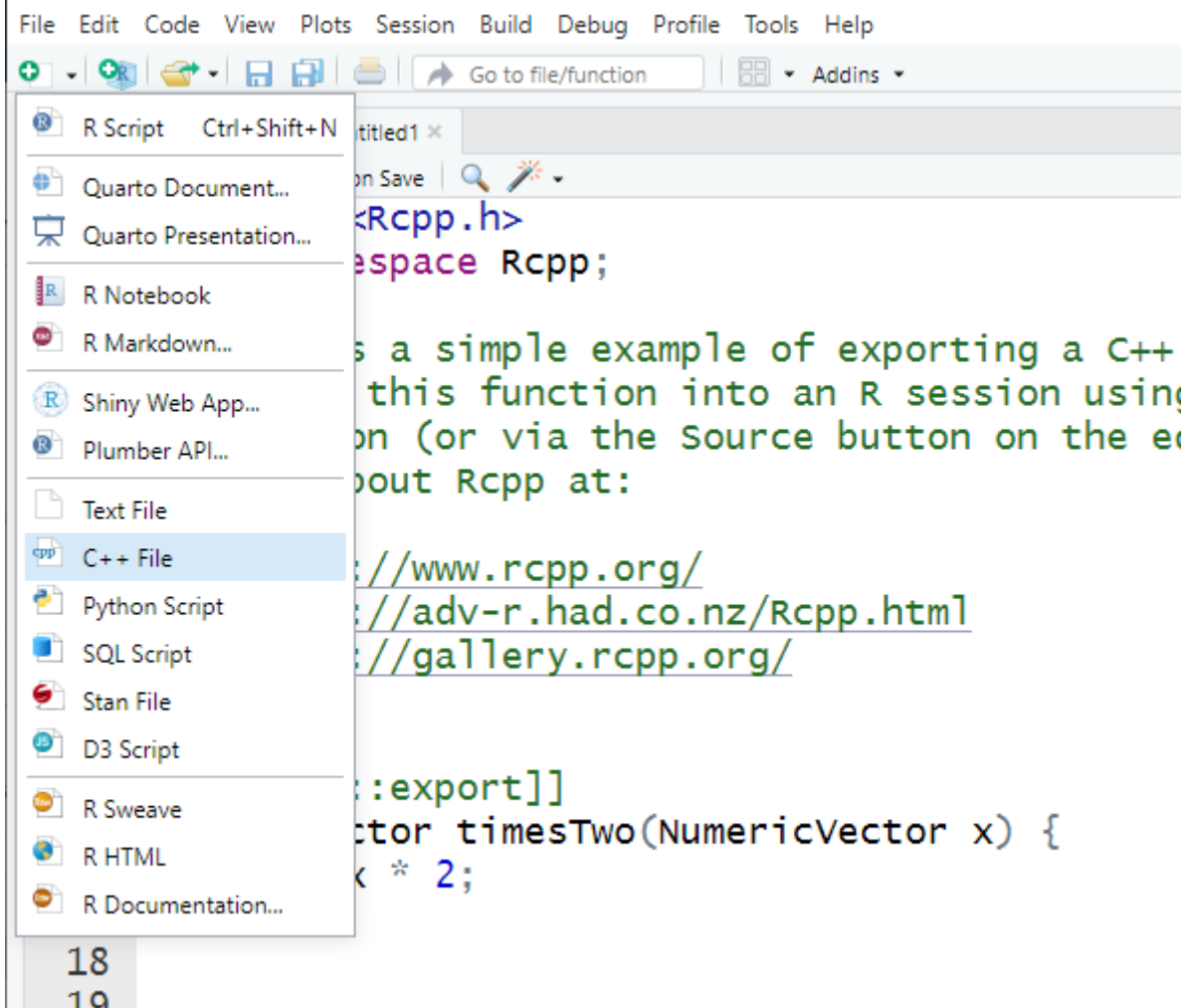
**WRITE C++ EASIER
WITH SOURCECPP()**



USING SOURCE C++ SCRIPTS

- ❖ Writing C++ code in character input of `cppFunction()` is simple, because everything is stored in the same script
- ❖ However, lack of code highlighting and syntax checker makes it very easy to make mistakes or spelling errors
- ❖ We can create C++ scripts in RStudio – which include syntax checking and code highlighting (files with *.cpp extension)
- ❖ This way more advanced codes can be created with more ease

CREATE NEW CPP SCRIPT



Begin the script with

```
#include <Rcpp.h>
```

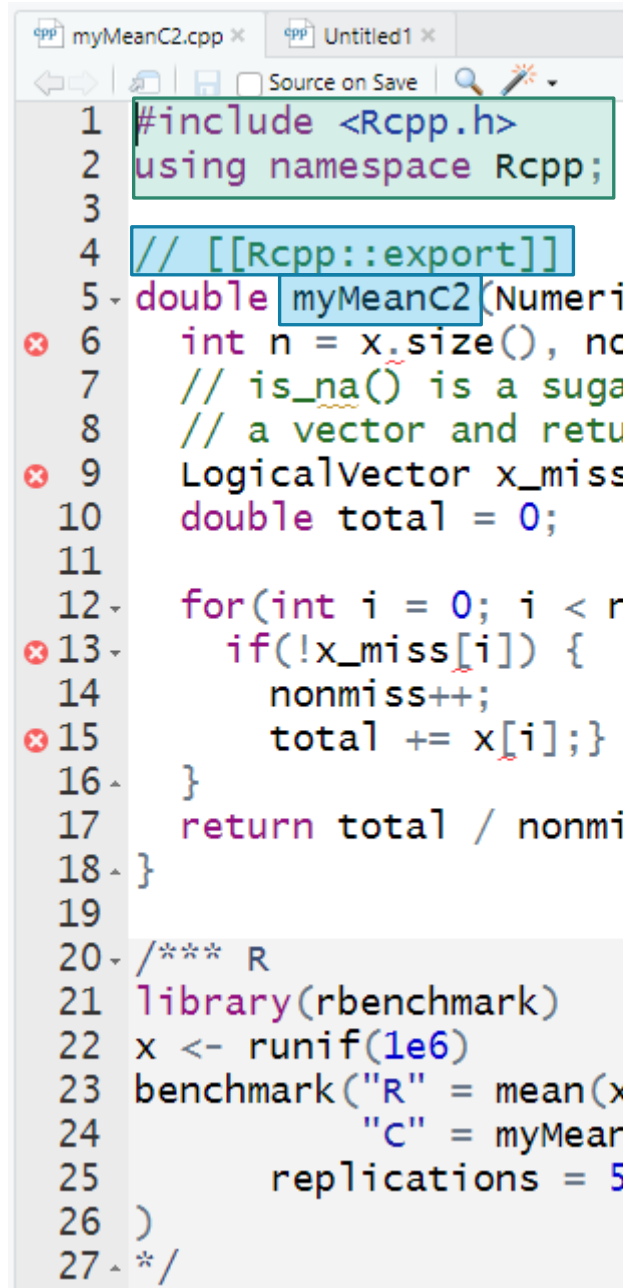
```
using namespace Rcpp;
```

to include Rcpp sugar and R-like classes

Put a special header just above the function definition to export it to R after running `sourceCpp()`

```
// [[Rcpp::export]]
```

SPACE AFTER `//` IS NECESSARY



```

1 #include <Rcpp.h>
2 using namespace Rcpp;
3
4 // [[Rcpp::export]]
5 double myMeanC2(NumericVector x) {
6     int n = x.size(), nonmiss = 0;
7     // is_na() is a sugar function, which takes
8     // a vector and returns a logical vector
9     LogicalVector x_miss = is_na(x);
10    double total = 0;
11
12    for(int i = 0; i < n; i++) {
13        if(!x_miss[i]) {
14            nonmiss++;
15            total += x[i];
16        }
17    }
18    return total / nonmiss;
19 }
20 /** R
21 library(rbenchmark)
22 x <- runif(1e6)
23 benchmark("R" = mean(x),
24           "C" = myMeanC2(x),
25           replications = 500
26 )
27 */

```

These elements are created by default in cpp scripts in RStudio

Rcpp::export so that myMeanC2 will be available in R

Run sourceCpp() on the file with C++ script to compile it and use the functions in R

```
sourceCpp("myMeanC2.cpp")
```

Begin the script with

```
#include <Rcpp.h>
```

```
using namespace Rcpp;
```

to include Rcpp sugar and R-like classes

Put a special header just above the function definition to export it to R after running sourceCpp()

```
// [[Rcpp::export]]
```

SPACE AFTER // IS NECESSARY

```

myMeanC2.cpp x  Untitled1 x
Source on Save
1 #include <Rcpp.h>
2 using namespace Rcpp;
3
4 // [[Rcpp::export]]
5 double myMeanC2(NumericVector x) {
6     int n = x.size(), nonmiss = 0;
7     // is_na() is a sugar function, which takes
8     // a vector and returns a logical vector
9     LogicalVector x_miss = is_na(x);
10    double total = 0;
11
12    for(int i = 0; i < n; i++) {
13        if(!x_miss[i]) {
14            nonmiss++;
15            total += x[i];}
16    }
17    return total / nonmiss;
18 }
19
20 /** R
21 library(rbenchmark)
22 x <- runif(1e6)
23 benchmark("R" = mean(x),
24           "C" = myMeanC2(x),
25           replications = 500
26 )
27 */

```

You can add R codes (for benchmarking or testing) within special code blocks – they will be evaluated when `sourceCpp()` comand will be run on the script

```

/** R
# your code

*/

```

Compilation of C++ code (and R code blocks within it) is done in R via calling `sourceCpp()` and the script name inside

```

#### IN R ####

sourceCpp("myMeanC2.cpp")

```